

#PACOTE COMPLETO COM SERVIDOR ORQUESTRADOR E FRONTEND

Vamos completar o pacote com o servidor orquestrador e o frontend. Preparei um `server.js` completo e funcional, com as integrações que faltavam.

##SIGA AS INSTRUÇÕES

Estrutura Final do Projeto

...

```
/eletromax-demo/  
├── seed.js          (já fornecido)  
├── catalogo.js     (já fornecido)  
├── server.js       (NOVO - orquestrador + frontend)  
├── package.json    (dependências)  
├── .env            (opcional, para variáveis)  
└── README.md      (instruções)  
...
```

1. `package.json`

```
```json  
{
 "name": "eletromax-demo",
 "version": "1.0.0",
 "description": "Assistente de dados para demo EletroMax",
 "main": "server.js",
 "scripts": {
 "start": "node server.js",
 "seed": "node seed.js"
 },
 "dependencies": {
 "@anthropic-ai/sdk": "^0.24.0",
 "@fastify/static": "^7.0.0",
 "dotenv": "^16.3.1",
 "fastify": "^4.25.0",
 "mysql2": "^3.6.0",
 "quickchart-js": "^3.1.0"
 },
 "devDependencies": {
 "@faker-js/faker": "^8.0.0"
 }
}
```
```

2. `server.js` (Completo)

```

````javascript
// server.js
require('dotenv').config();
const Fastify = require('fastify');
const fastifyStatic = require('@fastify/static');
const path = require('path');
const mysql = require('mysql2/promise');
const { QuickChart } = require('quickchart-js');
const { Anthropic } = require('@anthropic-ai/sdk');
const catalogo = require('./catalogo');

// ===== CONFIGURAÇÕES =====
const DB_CONFIG = {
 host: process.env.DB_HOST || 'localhost',
 user: process.env.DB_USER || 'root',
 password: process.env.DB_PASSWORD || 'sua_senha',
 database: process.env.DB_NAME || 'eletromax_demo'
};

// Se não tiver chave da Anthropic, use um stub (modo simulação)
const ANTHROPIC_API_KEY = process.env.ANTHROPIC_API_KEY || null;
let anthropic = null;
if (ANTHROPIC_API_KEY) {
 anthropic = new Anthropic({ apiKey: ANTHROPIC_API_KEY });
} else {
 console.warn('⚠ ANTHROPIC_API_KEY não definida. Usando modo simulação para extração de intenção.');
```

```
// ===== FUNÇÃO DE EXTRAÇÃO DE INTENÇÃO (com IA ou stub) =====
```

```
const SYSTEM_PROMPT_EXTRACAO = `
```

Você é um extrator de intenção para um assistente de dados empresariais (EletroMax Distribuidora).

Você deve analisar a pergunta do usuário e retornar APENAS um JSON válido com os campos:

- "intent": string (uma das opções abaixo)
- "params": objeto com os parâmetros extraídos

#### INTENÇÕES DISPONÍVEIS:

1. "estoque\_fornecedor" - Pergunta sobre estoque atual, entradas e quebra de um fornecedor específico.

Parâmetros: fornecedor\_nome (string), mes\_referencia (número, 1-12), ano\_referencia (número, 4 dígitos).

Exemplo: "Qual o estoque da Tramontina em julho?" ->

```
{"intent": "estoque_fornecedor", "params":
{"fornecedor_nome": "Tramontina", "mes_referencia": 7, "ano_referencia": 2026}}
```

2. "participacao\_fornecedor" - Pergunta sobre participação de compras de um grupo fornecedor (soma de todos os CNPJs).

Parâmetros: grupo\_nome (string), mes\_referencia, ano\_referencia.

Exemplo: "Qual a participação da Siemens nas compras de junho?" ->

```
{"intent": "participacao_fornecedor", "params":
{"grupo_nome": "Siemens", "mes_referencia": 6, "ano_referencia": 2026}}
```

3. "comparacao\_periodo" - Pergunta comparando dois períodos diferentes para um produto.

Parâmetros: produto\_nome (string), mes1, ano1, mes2, ano2.

Exemplo: "Compare o cabo 10mm em maio e junho" ->

```
{"intent": "comparacao_periodo", "params": {"produto_nome": "Cabo
10mm", "mes1": 5, "ano1": 2026, "mes2": 6, "ano2": 2026}}
```

4. "margem" - Pergunta sobre margem de lucro de um produto. (RESTRITO a financeiro)

Parâmetros: produto\_nome (string).

Exemplo: "Qual a margem do motor 5CV?" -> {"intent": "margem", "params":

```
{"produto_nome": "motor 5CV"}}
```

5. "fora\_escopo" - Quando a pergunta não se encaixa em nenhuma das acima (ex: "Qual o clima de hoje?").

Parâmetros: vazio.

#### REGRAS:

- Se o mês/ano não forem mencionados, use o mês e ano atuais (hoje: 20/07/2026, mes=7, ano=2026).

- Se mencionar "este mês", use 7/2026.

- Se mencionar "mês passado", use 6/2026.

- Retorne SOMENTE o JSON, sem texto adicional.

```
`;
`;
```

```
async function extrairIntencao(pergunta) {
```

```
 if (anthropic) {
```

```
 // Chamada real à Anthropic
```

```
 const response = await anthropic.messages.create({
```

```

 model: 'claude-3-5-haiku-20241022',
 max_tokens: 300,
 temperature: 0,
 system: SYSTEM_PROMPT_EXTRACAO,
 messages: [{ role: 'user', content: pergunta }]
 });
 const texto = response.content[0].text;
 try {
 return JSON.parse(texto);
 } catch (e) {
 console.error('Erro ao parsear JSON da IA:', texto);
 return { intent: 'fora_escopo', params: {} };
 }
} else {
 // MODO SIMULAÇÃO (stub) - extração simples por palavras-chave
 console.log(' 🤖 Usando stub de extração (sem IA)');
 const p = pergunta.toLowerCase();
 if (p.includes('estoque') || p.includes('quebra') || p.includes('entrou')) {
 // Tenta extrair fornecedor (palavra após "da", "do")
 let fornecedor = null;
 const match = pergunta.match(/(?:da|do|da |do)([A-Za-zÀ-ú]+)/i);
 if (match) fornecedor = match[1];
 if (!fornecedor) fornecedor = 'Tramontina'; // fallback
 return {
 intent: 'estoque_fornecedor',
 params: { fornecedor_nome: fornecedor, mes_referencia: 7, ano_referencia: 2026 }
 };
 } else if (p.includes('participação') || p.includes('participacao')) {
 const match = pergunta.match(/(?:da|do|da |do)([A-Za-zÀ-ú]+)/i);
 const grupo = match ? match[1] : 'Tramontina';
 return { intent: 'participacao_fornecedor', params: { grupo_nome: grupo, mes_referencia: 7,
ano_referencia: 2026 } };
 } else if (p.includes('compare') || p.includes('comparação') || p.includes('comparacao')) {
 // Pega o produto: entre "compare" e "em"
 const match = pergunta.match(/compare\s+([^\d]+?)\s+em/i);
 const produto = match ? match[1].trim() : 'Cabo';
 return { intent: 'comparacao_periodo', params: { produto_nome: produto, mes1: 5, ano1: 2026,
mes2: 6, ano2: 2026 } };
 } else if (p.includes('margem')) {
 const match = pergunta.match(/margem\s+do\s+([^\s]+)/i);
 const produto = match ? match[1] : 'motor 5CV';
 return { intent: 'margem', params: { produto_nome: produto } };
 } else {
 return { intent: 'fora_escopo', params: {} };
 }
}
}

// ===== FUNÇÃO PARA EXECUTAR CONSULTA DO CATÁLOGO =====
async function executarConsulta(intent, params, perfilUsuario) {
 // Busca a definição da consulta no catálogo
 const consultaDef = catalogo.find(c => c.id === intent);

```

```

if (!consultaDef) {
 throw new Error('Intenção não encontrada no catálogo.');
```

```

}

// 1. Verificar permissão
if (!consultaDef.permissoes.includes(perfilUsuario)) {
 throw new Error('PERMISSAO_NEGADA');
```

```

}

// 2. Preparar os parâmetros na ordem definida
const paramsOrder = consultaDef.params_def;
const valores = paramsOrder.map(key => params[key]);
// Verificar se todos os parâmetros estão presentes
for (const key of paramsOrder) {
 if (params[key] === undefined || params[key] === null) {
 throw new Error(`Parâmetro "${key}" não fornecido.`);
 }
}

// 3. Executar SQL (usando prepared statement)
const sql = consultaDef.sql;
const [rows] = await pool.query(sql, valores);
return rows;
}

// ===== FUNÇÃO PARA GERAR GRÁFICO (QuickChart) =====
function gerarGrafico(dados, titulo, tipo = 'bar', labels = null, datasets = null) {
 if (!dados || dados.length === 0) return null;
 const qc = new QuickChart();
 qc.setWidth(600);
 qc.setHeight(400);
 qc.setBackgroundColor('white');
```

```

 // Se não passou datasets, constrói automaticamente
 if (!datasets) {
 // Para estoque_fornecedor: espera colunas: produto_nome, total_entrado, saldo_atual,
 // quebra_absoluta
 if (dados[0].produto_nome) {
 const nomes = dados.map(r => r.produto_nome);
 const entradas = dados.map(r => r.total_entrado);
 const saldos = dados.map(r => r.saldo_atual);
 const quebras = dados.map(r => r.quebra_absoluta);
 qc.setConfig({
 type: 'bar',
 data: {
 labels: nomes,
 datasets: [
 { label: 'Entradas', data: entradas, backgroundColor: 'rgba(54, 162, 235, 0.6)' },
 { label: 'Saldo Atual', data: saldos, backgroundColor: 'rgba(75, 192, 192, 0.6)' },
 { label: 'Quebra', data: quebras, backgroundColor: 'rgba(255, 99, 132, 0.6)' }
]
 }
 },
 },

```

```

 options: { title: { display: true, text: titulo || 'Estoque vs Quebra' } }
 });
} else {
 // Fallback: gráfico genérico
 const keys = Object.keys(dados[0]);
 const primeiroKey = keys[0];
 const valores = dados.map(r => r[primeiroKey]);
 qc.setConfig({
 type: 'bar',
 data: {
 labels: dados.map((_, i) => `Item ${i+1}`),
 datasets: [{ label: primeiroKey, data: valores, backgroundColor: 'rgba(54, 162, 235, 0.6)' }]
 },
 options: { title: { display: true, text: titulo || 'Dados' } }
 });
}
} else {
 qc.setConfig({ type: tipo, data: { labels, datasets }, options: { title: { display: true, text:
titulo } } });
}

return qc.getUrl();
}

```

```

// ===== FUNÇÃO PARA ESTIMAR CUSTO (simples) =====
function estimarCusto(pergunta, resposta, tokensEstimados = null) {
 // Estimativa baseada no tamanho da pergunta + resposta
 const chars = pergunta.length + (resposta ? resposta.length : 0);
 const tokens = Math.ceil(chars / 4) + 20; // ~4 chars/token
 const custoUsd = (tokens / 1_000_000) * 1.0; // US$1/Mtok (Haiku input)
 const custoBrl = custoUsd * 5.6; // câmbio
 return {
 tokens: tokens,
 custoUsd: parseFloat(custoUsd.toFixed(6)),
 custoBrl: parseFloat(custoBrl.toFixed(4))
 };
}

```

```

// ===== ROTA PRINCIPAL - CHAT =====
app.post('/api/chat', async (request, reply) => {
 try {
 const { pergunta, perfil } = request.body;
 if (!pergunta || !perfil) {
 return reply.status(400).send({ error: 'Campos "pergunta" e "perfil" são obrigatórios.' });
 }
 }
}

```

```

// 1. Extrair intenção
const extracao = await extrairIntencao(pergunta);
console.log('🔍 Intenção extraída:', extracao);
const { intent, params } = extracao;

```

```

if (intent === 'fora_escopo') {

```

```

return reply.send({
 resposta: 'Desculpe, não entendi. Posso ajudar com estoque, fornecedores, comparações de
períodos ou margens (restrito).',
 alerta: null,
 grafico: null,
 custo: estimarCusto(pergunta, 'fora de escopo')
});
}

```

// 2. Executar consulta (com validação de permissão interna)

```

let rows;
try {
 rows = await executarConsulta(intent, params, perfil);
} catch (err) {
 if (err.message === 'PERMISSAO_NEGADA') {
 return reply.send({
 resposta: '❌ **Acesso Negado**. Você não tem permissão para acessar essa métrica.',
 alerta: null,
 grafico: null,
 custo: estimarCusto(pergunta, 'permissão negada')
 });
 }
 throw err; // outros erros
}

```

// 3. Gerar resposta formatada (aqui poderíamos usar IA para formatar, mas faremos template simples)

```

let respostaTexto = "";
let alerta = null;
let graficoUrl = null;

if (rows.length === 0) {
 respostaTexto = 'Nenhum dado encontrado para os critérios informados.';
} else {
 // Formatação básica para cada intent
 if (intent === 'estoque_fornecedor') {
 // Verifica se há quebra > 10%
 const maxQuebra = Math.max(...rows.map(r => parseFloat(r.quebra_percentual) || 0));
 if (maxQuebra > 10) {
 alerta = `⚠️ ALERTA DE QUEBRA DE ESTOQUE! A maior divergência é de $
{maxQuebra.toFixed(2)}%`;
 }
 // Monta tabela resumida
 let tabela = '| Produto | Entradas | Saldo | Quebra (%) |\n|-----|-----|-----|-----|\n';
 rows.slice(0, 10).forEach(r => {
 tabela += `| ${r.produto_nome} | ${r.total_entrado} | ${r.saldo_atual} | $
{r.quebra_percentual.toFixed(2)}% |\n`;
 });
 if (rows.length > 10) tabela += `| ... e mais ${rows.length - 10} produtos || |\n`;
 respostaTexto = `🇧🇷 **Estoque e Quebra por Fornecedor**\n\n${tabela}`;
 // Gráfico
 graficoUrl = gerarGrafico(rows, 'Entradas vs Saldo vs Quebra');
 }
}

```

```

 } else if (intent === 'participacao_fornecedor') {
 let tabela = '| Fornecedor (CNPJ) | Total Comprado | Participação % |\n|-----|-----|-----|\n';
 rows.forEach(r => {
 tabela += `| ${r.fornecedor} (${r.cnpj}) | R$ ${r.total_comprado.toFixed(2)} | $
 {r.participacao_percentual.toFixed(2)}% |\n`;
 });
 respostaTexto = ` 🍷 **Participação do Grupo Fornecedor**\n\n${tabela}`;
 // Gráfico de pizza ou barras
 const labels = rows.map(r => r.fornecedor);
 const data = rows.map(r => r.total_comprado);
 graficoUrl = gerarGrafico(null, 'Participação por CNPJ', 'pie', labels, [{ data, backgroundColor:
 ['#FF6384', '#36A2EB', '#FFCE56', '#4BC0C0', '#9966FF'] }]);
 } else if (intent === 'comparacao_periodo') {
 const p1 = rows[0];
 const p2 = rows[1];
 const variacao = p2 ? ((p2.total - p1.total) / p1.total * 100).toFixed(2) : 0;
 respostaTexto = ` 📊 **Comparação de Períodos**\n\n` +
 ` - Período 1: ${p1.total} unidades\n` +
 ` - Período 2: ${p2 ? p2.total : 0} unidades\n` +
 ` - Variação: ${variacao}%`;
 // Gráfico de barras
 graficoUrl = gerarGrafico(null, 'Comparação', 'bar', ['Período 1', 'Período 2'], [{ data: [p1.total,
 p2 ? p2.total : 0], backgroundColor: ['#36A2EB', '#FF6384'] }]);
 } else if (intent === 'margem') {
 let tabela = '| Produto | Custo Médio | Preço Venda | Margem Unit. | Margem % |\n|-----|-----|-----|-----|-----|\n';
 rows.forEach(r => {
 tabela += `| ${r.produto} | R$ ${r.custo_medio.toFixed(2)} | R$ ${r.preco_venda.toFixed(2)}
 | R$ ${r.margem_bruta_unit.toFixed(2)} | ${r.margem_percentual.toFixed(2)}% |\n`;
 });
 respostaTexto = ` 💰 **Margem por Produto**\n\n${tabela}`;
 // Gráfico
 const labels = rows.map(r => r.produto);
 const margens = rows.map(r => r.margem_percentual);
 graficoUrl = gerarGrafico(null, 'Margem %', 'bar', labels, [{ label: 'Margem %', data: margens,
 backgroundColor: 'rgba(75, 192, 192, 0.6)' }]);
 } else {
 respostaTexto = 'Dados obtidos:\n' + JSON.stringify(rows, null, 2);
 }
 }

 // 4. Estimar custo
 const custo = estimarCusto(pergunta, respostaTexto);

 // 5. Resposta final
 return reply.send({
 resposta: respostaTexto,
 alerta: alerta,
 grafico: graficoUrl,
 custo: custo
 });

```



```
 } catch (error) {
 console.error('Erro no /api/chat:', error);
 return reply.status(500).send({ error: 'Ocorreu um erro ao processar sua pergunta. Tente novamente.' });
 }
 });
```

```
// ===== ROTA PARA SERVIÇO DE ESTÁTICOS (frontend) =====
// Vamos servir o HTML diretamente na raiz
app.get('/', async (request, reply) => {
 const html = `
<!DOCTYPE html>
<html lang="pt-BR">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>EletroMax Assistente</title>
 <style>
 * { box-sizing: border-box; margin: 0; padding: 0; }
 body {
 font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
 background: #f4f7fa;
 display: flex;
 justify-content: center;
 align-items: center;
 height: 100vh;
 margin: 0;
 }
 .chat-container {
 width: 100%;
 max-width: 800px;
 height: 90vh;
 background: white;
 border-radius: 20px;
 box-shadow: 0 10px 30px rgba(0,0,0,0.15);
 display: flex;
 flex-direction: column;
 overflow: hidden;
 }
 .header {
 background: #1a2a3a;
 color: white;
 padding: 20px;
 display: flex;
 justify-content: space-between;
 align-items: center;
 border-bottom: 3px solid #f9a825;
 }
 .header h1 { font-size: 1.3rem; font-weight: 500; }
 .header .perfil-selector {
 display: flex;

```

```
gap: 10px;
background: #2c3e50;
padding: 5px 10px;
border-radius: 30px;
}
.header .perfil-selector button {
background: transparent;
border: none;
color: #aaa;
padding: 6px 15px;
border-radius: 20px;
cursor: pointer;
font-weight: 600;
transition: 0.2s;
}
.header .perfil-selector button.active {
background: #f9a825;
color: #1a2a3a;
}
.header .perfil-selector button:hover { color: white; }
.messages {
flex: 1;
padding: 20px;
overflow-y: auto;
background: #f0f2f5;
}
.message {
margin-bottom: 15px;
display: flex;
flex-direction: column;
}
.message.user {
align-items: flex-end;
}
.message.bot {
align-items: flex-start;
}
.message .bubble {
max-width: 75%;
padding: 12px 18px;
border-radius: 18px;
word-wrap: break-word;
line-height: 1.5;
font-size: 0.95rem;
}
.message.user .bubble {
background: #1a2a3a;
color: white;
border-bottom-right-radius: 4px;
}
.message.bot .bubble {
background: white;
```

```
color: #222;
border-bottom-left-radius: 4px;
box-shadow: 0 2px 5px rgba(0,0,0,0.05);
}
.message .bubble table {
border-collapse: collapse;
margin: 8px 0;
font-size: 0.85rem;
}
.message .bubble th, .message .bubble td {
border: 1px solid #ddd;
padding: 5px 8px;
text-align: left;
}
.message .bubble th { background: #e9ecef; }
.message .bubble .alerta {
background: #fff3cd;
border-left: 6px solid #ffc107;
padding: 10px;
margin: 10px 0;
border-radius: 4px;
font-weight: bold;
color: #856404;
}
.message .bubble img {
max-width: 100%;
border-radius: 8px;
margin-top: 10px;
}
.message .bubble .custo {
font-size: 0.75rem;
color: #666;
margin-top: 8px;
border-top: 1px dashed #ddd;
padding-top: 6px;
text-align: right;
}
.input-area {
padding: 15px 20px;
background: white;
border-top: 1px solid #e0e0e0;
display: flex;
gap: 10px;
}
.input-area input {
flex: 1;
padding: 12px 18px;
border: 1px solid #ccc;
border-radius: 30px;
font-size: 1rem;
outline: none;
}
```

```

.input-area input:focus { border-color: #1a2a3a; }
.input-area button {
 background: #f9a825;
 border: none;
 padding: 0 25px;
 border-radius: 30px;
 font-weight: 600;
 color: #1a2a3a;
 cursor: pointer;
 transition: 0.2s;
}
.input-area button:hover { background: #fbc02d; }
.sugestoes {
 display: flex;
 gap: 8px;
 flex-wrap: wrap;
 padding: 8px 20px;
 background: #fafafa;
 border-top: 1px solid #eee;
}
.sugestoes button {
 background: #e9ecef;
 border: none;
 padding: 5px 12px;
 border-radius: 15px;
 font-size: 0.8rem;
 cursor: pointer;
 color: #333;
 transition: 0.2s;
}
.sugestoes button:hover { background: #d0d7dd; }
.loading {
 display: inline-block;
 width: 20px;
 height: 20px;
 border: 3px solid #f3f3f3;
 border-top: 3px solid #f9a825;
 border-radius: 50%;
 animation: spin 0.8s linear infinite;
 margin: 5px 0;
}
@keyframes spin { to { transform: rotate(360deg); } }
@media (max-width: 600px) {
 .header h1 { font-size: 1rem; }
 .header .perfil-selector button { padding: 4px 10px; font-size: 0.8rem; }
 .message .bubble { max-width: 90%; }
}
</style>
</head>
<body>
<div class="chat-container">
 <div class="header">

```

```

<h1> ⚡ EletroMax Assistente</h1>
<div class="perfil-selector">
 <button class="active" data-perfil="compras">Compras</button>
 <button data-perfil="financeiro">Financeiro</button>
</div>
</div>
<div class="messages" id="messages"></div>
<div class="sugestoes">
 <button data-pergunta="Qual o estoque e quebra da Tramontina neste mês?"> 🔍
Tramontina</button>
 <button data-pergunta="Qual a participação da Tramontina nas compras deste mês?"> 🇧🇷
Participação</button>
 <button data-pergunta="Compare o Disjuntor 16A em maio e junho."> 📅 Comparar</button>
 <button data-pergunta="Qual a margem do motor 5CV?"> 💰 Margem</button>
</div>
<div class="input-area">
 <input type="text" id="input" placeholder="Digite sua pergunta..." />
 <button id="send">Enviar</button>
</div>
</div>

<script>
const messagesDiv = document.getElementById('messages');
const input = document.getElementById('input');
const sendBtn = document.getElementById('send');
let perfilAtual = 'compras';

// Seleção de perfil
document.querySelectorAll('.perfil-selector button').forEach(btn => {
 btn.addEventListener('click', function() {
 document.querySelectorAll('.perfil-selector button').forEach(b => b.classList.remove('active'));
 this.classList.add('active');
 perfilAtual = this.dataset.perfil;
 addMessage('bot', ' 🧑 Perfil alterado para **' + perfilAtual + '**.');
 });
});

// Sugestões
document.querySelectorAll('.sugestoes button').forEach(btn => {
 btn.addEventListener('click', function() {
 input.value = this.dataset.pergunta;
 enviar();
 });
});

function addMessage(tipo, conteudo) {
 const div = document.createElement('div');
 div.className = 'message ' + tipo;
 const bubble = document.createElement('div');
 bubble.className = 'bubble';
 // Converte markdown simples (negrito, tabelas etc)
 bubble.innerHTML = conteudo.replace(/*(.*?)*/g, '$1');
}

```

```

div.appendChild(bubble);
messagesDiv.appendChild(div);
messagesDiv.scrollTop = messagesDiv.scrollHeight;
}

```

```

function addLoading() {
 const div = document.createElement('div');
 div.className = 'message bot';
 div.id = 'loading';
 const bubble = document.createElement('div');
 bubble.className = 'bubble';
 bubble.innerHTML = '<div class="loading"></div>';
 div.appendChild(bubble);
 messagesDiv.appendChild(div);
 messagesDiv.scrollTop = messagesDiv.scrollHeight;
}

```

```

function removeLoading() {
 const loading = document.getElementById('loading');
 if (loading) loading.remove();
}

```

```

async function enviar() {
 const pergunta = input.value.trim();
 if (!pergunta) return;
 input.value = "";
 addMessage('user', pergunta);
 addLoading();

```

```

 try {
 const response = await fetch('/api/chat', {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({ pergunta, perfil: perfilAtual })
 });
 const data = await response.json();
 removeLoading();

```

```

 if (data.error) {
 addMessage('bot', '✖ Erro: ' + data.error);
 return;
 }

```

```

 let html = "";
 if (data.resposta) {
 html += data.resposta.replace(/\n/g, '
');
 }
 if (data.alerta) {
 html += '<div class="alerta">' + data.alerta + '</div>';
 }
 if (data.grafico) {
 html += '';

```

```

 }
 if (data.custo) {
 html += '<div class="custo"> 💰 Custo estimado: R$ ' + data.custo.custoBrl.toFixed(4) + ' (' +
data.custo.tokens + ' tokens)</div>';
 }
 addMessage('bot', html);
 } catch (err) {
 removeLoading();
 addMessage('bot', ' ❌ Erro de rede. Tente novamente. ');
 console.error(err);
 }
}

```

```

sendBtn.addEventListener('click', enviar);
input.addEventListener('keypress', (e) => { if (e.key === 'Enter') enviar(); });

```

```

// Mensagem de boas-vindas
addMessage('bot', ' 🙋 Olá! Sou o assistente da EletroMax. Faça perguntas sobre estoque,
fornecedores ou margens. Escolha seu perfil e pergunte!');
</script>
</body>
</html>
`;
reply.type('text/html').send(html);
});

```

```

// ===== INICIAR SERVIDOR =====
const PORT = process.env.PORT || 3000;
app.listen({ port: PORT, host: '0.0.0.0' }, (err, address) => {
 if (err) {
 console.error(err);
 process.exit(1);
 }
 console.log(` 🚀 Servidor rodando em ${address} `);
 console.log(` 🌐 Acesse no navegador: http://localhost:${PORT} `);
});
...

```

---

## 3. Arquivo `.env` (opcional)

```

...
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=minhasenha
DB_NAME=eletromax_demo
ANTHROPIC_API_KEY=sk-ant-...
PORT=3000
...

```

---

## ## 4. Instruções Finais

1. **Crie a pasta `public/`** (vazia, pois o HTML já está inline, mas pode deixar para futuros assets).
2. **Instale as dependências:**

```
```bash
npm install
```
```
3. **Rode o seed** (cria a base e os dados):

```
```bash
npm run seed
```
```
4. **Inicie o servidor:**

```
```bash
npm start
```
```
5. Abra o navegador em `http://localhost:3000`.

---

## ### Funcionalidades incluídas:

- ☒ Frontend com chat, seletor de perfil e sugestões de perguntas.
- ☒ Orquestrador que extrai intenção (com suporte a IA real e stub de simulação).
- ☒ Catálogo com permissões e execução SQL.
- ☒ Geração de gráficos via QuickChart.
- ☒ Alerta de quebra > 10%.
- ☒ Custo estimado por conversa.
- ☒ Tratamento de erros e recusas de permissão.

---

Agora você tem **tudo** para rodar a demo. Se quiser, posso também criar um script de testes automatizados ou um modo "apresentação" que executa o roteiro automaticamente. Fique à vontade para pedir mais ajustes. Boa sorte na apresentação! 🚀